Security Reference Architecture for AI-optimized Data Mirrors

Request for comments

AJS Consult LLC. Editor: Aaron Soellinger.

Contents

1 . . . 5

2 Updates . . . 7

2.1 **Follow the review** . . . 7

2.2 **All updates** . . . 7

3 Document control . . . 9

3.1 **Submission phases** . . . 9

4 Executive summary . . . 11

5 Contents . . . 13

6 What it defends against . . . 15

7 System Requirements . . . 17

8 The design on one page . . . 19

9 How it works . . . 21

9.1 **Key management lifecycle** . . . 21

9.1.1 Create . . . 22

9.1.2 Read . . . 23

9.1.3 Update . . . 24

9.1.4 Expire . . . 25

9.1.5 Delete . . . 26

9.2 **How data moves** . . . 27

9.2.1 Collect and push . . . 27

9.2.2 Approve a reader or writer . . . 28

9.2.3 Search and retrieve . . . 29

9.2.4 Update . . . 30

9.2.5 Revoke and recover . . . 31

10 Latency budget . . . 33

11 What it does not protect . . . 35

12 Decisions made to simplify . . . 37

13 Questions for reviewers . . . 39

Appendices . . . 41

A Appendix A. Single or multi-tenant . . . 43

1.

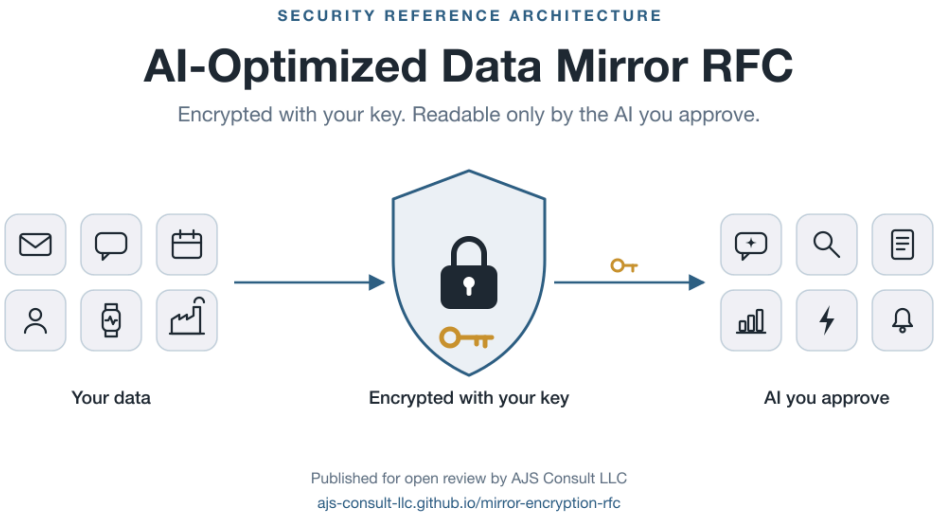


Figure 1.1: AI-Optimized Data Mirror RFC: your data, encrypted with your key, readable only by the AI you approve

> **Note**
>
> **Request for comments. Round 1 is open until 11 December 2026, 23:59 UTC.**

What this is. A security reference architecture for AI-optimized data mirrors: a private copy of a person's messages and data that only the readers they choose, including their AI assistants, can search. AJS Consult LLC publishes it for open review.

How to take part

1. **Read it.** Start with the executive summary. The download button in the menu gives a PDF.
2. **Comment.** In the review copy, signed in to a Google account. Its link is posted on the Updates page when the round opens.
3. **Follow the review.** Updates: sign up by email, use the feed, or watch on GitHub.
4. **Found a security problem in the running service?** Email aaron@matterboard.ai with SECURITY in the subject. Never post it in a comment.

Round 1	
Opens	11 November 2026
Comments close	11 December 2026, 23:59 UTC
Decisions published	By 25 December 2026

Nov 11, 2026 · @aaron soellinger

A request for comments (RFC) on encrypting private data mirrors so that a copy of the stored data is useless on its own, and the data opens only while a device or AI the user approved is making a request.

2. Updates

2.1 Follow the review

- **By email.** Sign up for review-window notices: when a round opens, its midpoint, when it closes, and when decisions are published.
- **By feed.** releases.atom, in any feed reader.
- **On GitHub.** Watch the repository, then choose Custom and Releases.

Email addresses are used only for these notices, never shared, and deleted when the review ends. Every email carries an unsubscribe link.

2.2 All updates

No updates yet.

3. Document control

Field	Value
Status	Request for comments. A design proposal.
Version	0.1, 11 November 2026
Owner and editor	Owned by AJS Consult LLC, © 2026 AJS Consult LLC. Edited by Aaron Soellinger.
License	CC BY 4.0. You may share, quote and adapt it, with credit to AJS Consult LLC.
Maintained by	Aaron Soellinger,
How it was written	Drafted from a read of private source code on 11 November 2026. Manually reviewed, the author is personally responsible for every claim in it.
Commitment	Nothing here commits AJS Consult LLC until it ships and the company's privacy policy says so.
Version of record	The copy at this address. Each revision is dated and listed below, and earlier versions stay readable in the document's history.
Disclosures	The editor founded and owns AJS Consult LLC, which owns the brands PRCVD.AI, a machine learning consultancy, and MatterBoard, a safe, secure and private messaging mirror optimized for AI tool use. AJS Consult LLC is actively engaged in commercial programs to safely, securely and privately manage data in AI pipelines for security, privacy and safety critical industries.

Comments. Anyone may and should comment, on any part of this document. Your comments stay yours. By posting one, you let the author use the idea without obligation or payment, and you are credited by name if you ask. Please leave out confidential information and anything about other people.

Security problems in the running service go privately to the contact in section 17 of the v0.1 tab, never in a comment here.

Open question: the date the comment period closes.

Version	Date	Change
0.1	11 November 2026	Initial commit.

3.1 Submission phases

Review runs in rounds. Each round is seven days, six for review and one for editing, and the rounds repeat until every comment is addressed.

1. **Review, 6 days.** The current version is open, and anyone may comment on any part of it.
2. **Editing, 1 day.** No new comments are taken. The editor addresses every comment in its own thread: accepted and changed, declined with a reason, or carried into Questions for reviewers.
3. **New version.** The revised version is published and added to the version table above, with what changed. The next review opens on it.

A comment that arrives on an editing day counts toward the next round. The rounds end when a review closes with no new comments and none left unaddressed; that version is then marked final.

Round	Version	Review opens (6 days)	Editing day (1 day)	Next version
1	0.1	To be set	To be set	0.2
2	0.2	The day after round 1′s editing day	6 days after the review opens	0.3
Later rounds	As needed	The same pattern	The same pattern	Final, when no comments remain

4. Executive summary

Today, AI assistants connect directly to a user's private correspondence via a variety of partially and fully supported, secure (and insecure) connectors. Under this status quo, AI providers become the custodians of this data, to the end of enabling bleeding edge automation and research capabilities to their users. Simply put, users should not need to hand over custody of their data in exchange for access to the best AIs. This document proposes a security architecture for a messaging mirror that can be securely, safely and privately operated by the user themselves or by a trusted 3rd party.

The proposal. One random key is made on the user's own computer, and messages are encrypted there before they are uploaded. Each device or AI the user approves gets its own token, and the server can open the data only while one of them is making a request, so a copy of the database alone is useless. This is not end-to-end encryption: the server sees what it serves while it serves it, and What it does not protect lists the limits.

The problem. Data ownership concerns are among the most significant impediments to the proliferation of beneficial AI, especially in security, safety and privacy critical use cases. There currently exists no complete security reference architecture to enable individuals or organizations to securely, safely and privately host their own data mirrors in a way that is integrated and compatible with AI assistants. Closed-source solutions to data mirroring require users to trust the implementation, and this document is meant to align users on "the right way" to achieve this goal so trust can be attained.

The alternative. Without a community effort to develop a security, privacy and safety standard around private data, the most likely outcome is for AIs to become more entrenched in their position as system of record for user's private data optimally stored for use by AI. Private 3rd party companies hosting closed source messaging mirrors require the user to trust that bond, which is a non-starter for anyone hosting sensitive or mission critical data. As well, the technical requirements of building such data management systems require seasoned, credentialed software engineering veterans and are thus costly and/or out of reach for most potential users of powerful AI.

What is asked of you. Help us figure out what's worth building that will actually get adopted because it is trusted. Attack the design. What it does not protect, the search leakage (question 1) and the questions for reviewers matter most.

5. Contents

1. Document control
2. Executive summary
3. What it defends against
4. System Requirements
5. The design on one page
6. How it works
   a. Key management lifecycle
      - Create
      - Read
      - Update
      - Expire
      - Delete
   b. How data moves
      - Collect and push
      - Approve a reader or writer
      - Search and retrieve
      - Update
      - Revoke and recover
7. Latency budget
8. What it does not protect
9. Decisions made to simplify
10. Questions for reviewers

Appendix A. Single or multi-tenant

6. What it defends against

The design has one goal: a copy of the stored data is useless without a credential the user approved. It does not hide data from the server while the server answers an approved request.

Who	Protected?
Someone who steals the database, a backup or a disk	Yes. They hold ciphertext, search tags and token hashes, and no key. The tags still leak some structure (see What it does not protect). Attachments are encrypted by the storage service under a key it does not keep, so a copy of the bucket is unreadable as well.
The operator's staff reading stored data	Yes, in the same way
A reader or writer after the user revokes it	Yes, through the service: its token no longer finds a copy
A compromised server, or an operator who changes the server's code	No. It sees the account key during any approved request and could keep it.
Malware on the user's desktop	No. The desktop holds the account key.

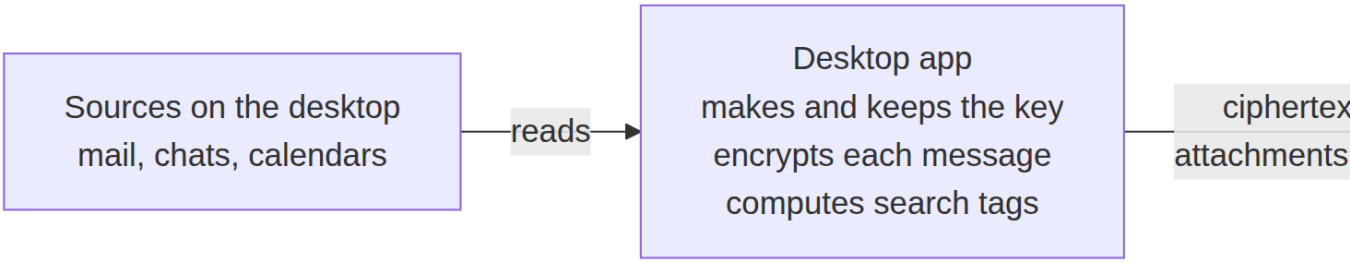
7. System Requirements

The design exists to meet five requirements, and nothing in it serves anything else.

#	Requirement	How the design meets it
R1	The user controls the keys	The account key is made on the user's desktop. A reader or writer gets access only when the user approves it, and loses it when the user revokes it.
R2	The desktop sync app collects and pushes messages in real time, encrypted from the source	The app encrypts each message under a content key derived from the account key, and computes its search tags, before it uploads. The server never receives plaintext from the app. Attachments are the exception (see Decisions made to simplify).
R3	Data is secure at rest in the database	The database holds ciphertext, keyed tags and token hashes. Nothing in it opens without a token.
R4	Approved readers and writers can search, retrieve and update in near real time	Each holds a token that opens its own copy of the account key for one request. A message is searchable as soon as it is stored.
R5	Under 10 seconds from a user's question to the AI's first response	The mirror's share is budgeted at 1 second at the 95th percentile. The rest belongs to the AI.

8. The design on one page

There is one key, one kind of token, and two algorithms: AES-256-GCM encrypts everything, and SHA-256 does the rest as a hash, as HMAC and as HKDF. Everything else follows from them.



- **One key.** On installation, the desktop sync application asks the operating system for 256 random bits, and those bits are the “account key”. The user’s OS key store holds the account key and releases it only to approved application(s). The account key never encrypts anything itself. Instead, HKDF-SHA256 turns it into two working keys, one per job: (1) a content key that encrypts each message (AES-256-GCM) and (2) a tag key that computes the search tags (HMAC-SHA256). A third, the storage key, is derived the same way for attachments. They are re-derived whenever they are needed, so nothing extra is stored, and learning one working key reveals neither the other nor the account key. Because both come from the one account key, a reader that can open its copy of the account key can both read messages and search them.

Why this is current best practice: a full-strength random key from an approved generator (NIST SP 800-133 Rev. 2); one key per purpose, never one key for two jobs (NIST SP 800-57 Part 1 Rev. 5, key separation); and working keys derived with HKDF (RFC 5869), the extract-then-expand construction NIST approves in SP 800-56C Rev. 2 and the one TLS 1.3 uses for all its keys (RFC 8446).

- **Encrypted at the source.** The desktop app encrypts each message with AES-256-GCM, bound to the account and the message's id, and computes its search tags with HMAC-SHA256. Only ciphertext and tags leave the desktop. Attachments are the exception: the desktop uploads them as they are, and the storage service encrypts each one under a storage key that is derived from the account key, supplied with the request and never kept (see Decisions made to simplify).

  Why this is current best practice: AES-256-GCM is the NIST-approved authenticated encryption mode (SP 800-38D, over AES in FIPS 197), so tampering is detected, not just hidden. Binding each ciphertext to its account and message id as associated data follows the AEAD design in RFC 5116, which stops a stored ciphertext being swapped into another row or account. HMAC-SHA256 is the approved keyed hash (FIPS 198-1), and keyed tags over fragments are the classic construction for search over encrypted data (Song, Wagner and Perrig, IEEE S&P 2000). Its known limit, that equal fragments produce equal tags, is question 1, and GCM's nonce limit is question 2.

- **One kind of token.** Every reader or writer the user approves gets a random 256-bit token: a connected AI, the web app, a second device, a script. The server keeps the token's SHA-256, to find it, and one copy of the account key locked under a key derived from the token (AES-256-GCM, the cipher that also encrypts messages). One token, one copy.

  Why this is current best practice: high-entropy bearer tokens are the OAuth standard (RFC 6750), and short lifetimes with refresh-token rotation follow the OAuth security best current practice (RFC 9700). Keeping only a hash of each token means a database copy yields no usable credential, the same rule NIST SP 800-63B sets for stored passwords. Locking one copy of the account key per holder with AES-256-GCM, an authenticated cipher that NIST also approves for wrapping keys (NIST SP 800-38F) is envelope encryption, the pattern cloud key-management services use.

- **Search without plaintext.** Inside an approved request the server unlocks the account key, turns the query into tags, finds candidates in an ordinary database index, and decrypts only those.

  Why this is current best practice: keyed tags are the standard construction for search over encrypted data (Song, Wagner and Perrig, IEEE S&P 2000). Filtering on trigrams and then checking the real match is how production regex search works (Russ Cox, Regular Expression Matching with a Trigram Index). The known cost is access-pattern leakage (Cash et al., Leakage-Abuse Attacks Against Searchable Encryption, CCS 2015), which is why this design names it as a limit and asks reviewers about it (question 1).

- **Revocation is deletion.** Deleting a token's hash and its copy cuts that reader or writer off at once.

  Why this is current best practice: revoking one holder deletes its only way to open the key through the service. Deleting every copy, as account deletion does, is cryptographic erase, a sanitization method NIST recognizes (SP 800-88 Rev. 1). Revoking a token server-side, effective on its next use, is what OAuth token revocation specifies (RFC 7009). Two gaps remain: backups hold old copies for up to 30 days, and the other copies of the same key stay valid (see What it does not protect).

The server never stores the account key, a token or plaintext. It holds the account key in memory only while it answers a request from an approved token.

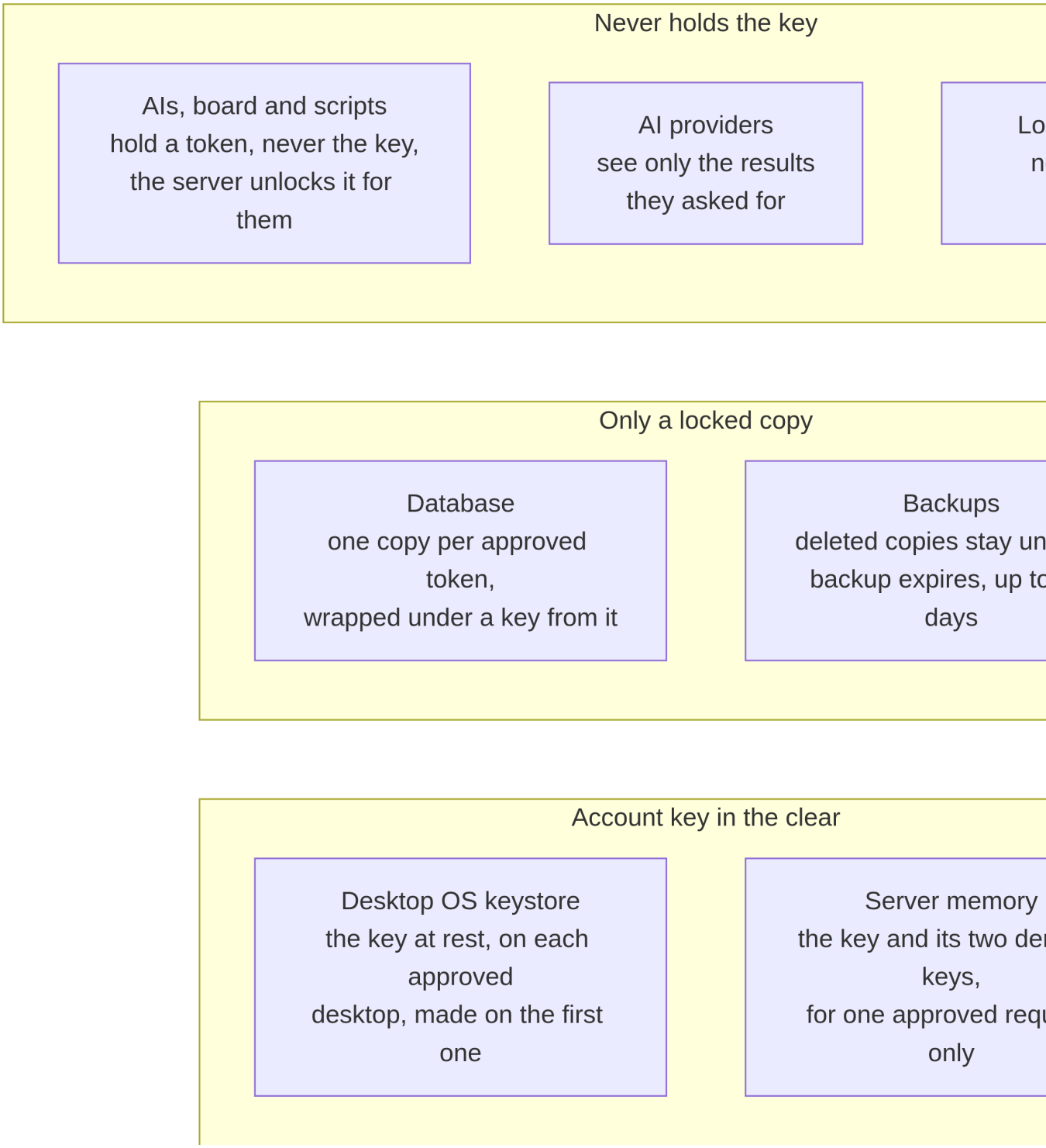
One rule for every credential. Each credential has its own locked copy of the account key. Every hand-off opens the copy with the old credential, locks a new copy under the new one, and deletes the old. An OAuth authorization code, the web app's connect code and a renewed token are all hand-offs of this kind, so the server never needs to store a token while one is in transit. A recovery code is the same thing on paper.

9. How it works

The same machinery seen twice: first what happens to the account key, then what happens to the data.

9.1 Key management lifecycle

The account key is created once on the desktop, opened only inside approved requests, changed only by adding or removing locked copies, and destroyed by deleting every copy.

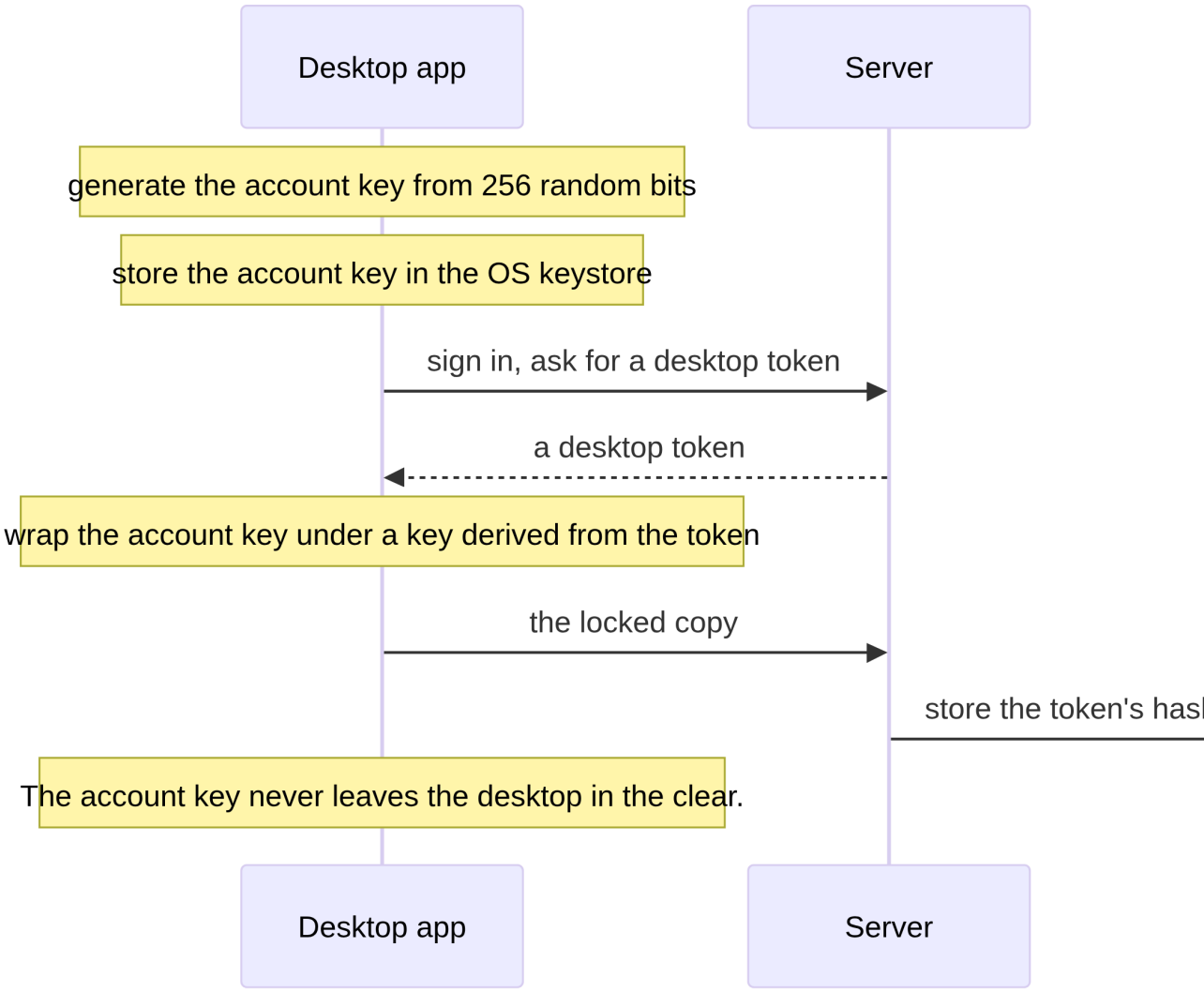


Only a desktop keystore holds the account key at rest. The server holds it in memory for one request at a time; everything else holds a locked copy, a token, or nothing.

Every change to who can read the mirror is a copy of the account key added or deleted; the account key itself changes only if it is rotated.

9.1.1 Create

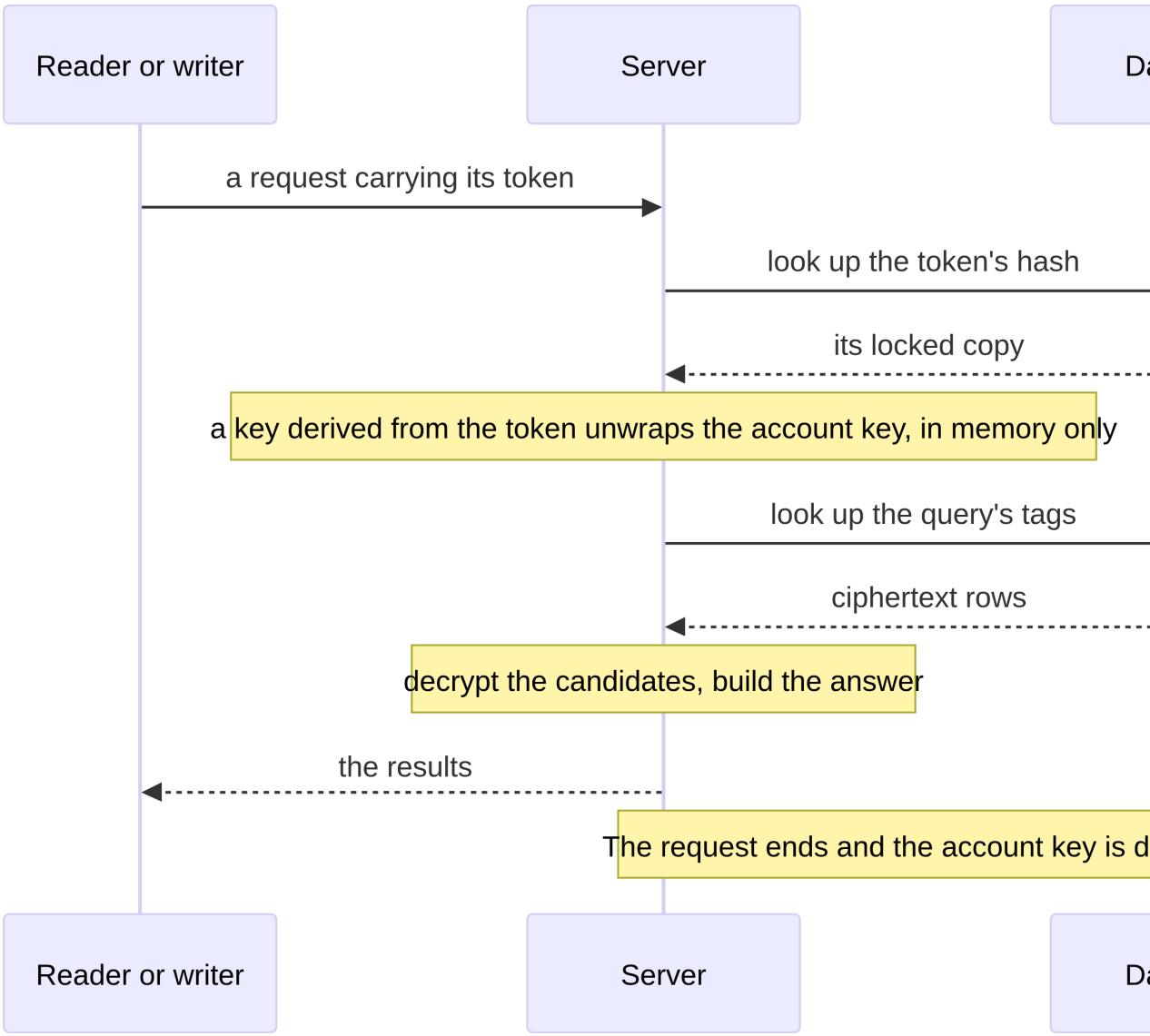
On first run, the desktop app generates the account key from 256 random bits and stores it in the OS keystore. It then signs in, receives its own desktop token, wraps the account key under a key derived from that token, and uploads only that locked copy. The server stores the copy beside the token's hash; it never sees the account key and never keeps the token.



Why this is best practice. The key is made from a cryptographically secure random source on the device that will hold it, which is what NIST SP 800-133 Rev. 2 recommends, and it is stored in the OS key store, hardware-backed where the platform offers it, rather than in a file. The server receives only a wrapped copy (NIST SP 800-38F), so nothing the server stores can open the data. That separation is the core idea of client-side and envelope encryption.

9.1.2 Read

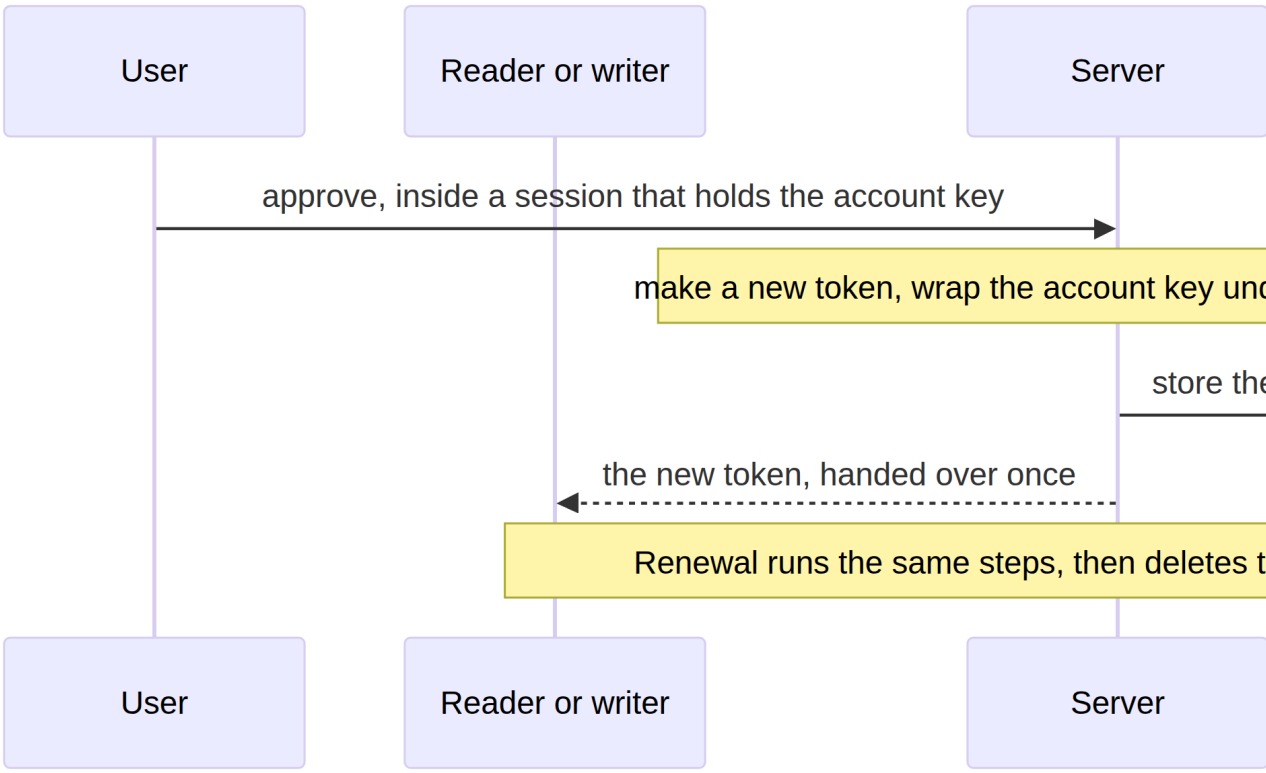
A reader's request carries its token. The server finds the reader's locked copy by the token's hash, derives the unwrapping key from the token with HKDF, and opens the account key in memory. It looks up the query's tags, decrypts only the candidate rows and returns the results. When the request ends, the account key is dropped; nothing of it is written to disk or logs.



Why this is best practice. The key is opened only in memory, only for one request, and then dropped. That follows NIST SP 800-57 Part 1: keep plaintext keys out of persistent storage and out of logs, and limit how long they exist. The unwrapping key is derived from the token with HKDF (RFC 5869), the standard way to turn a high-entropy secret into a key, and the server looks tokens up by hash, so its stored data holds no usable credential.

9.1.3 Update

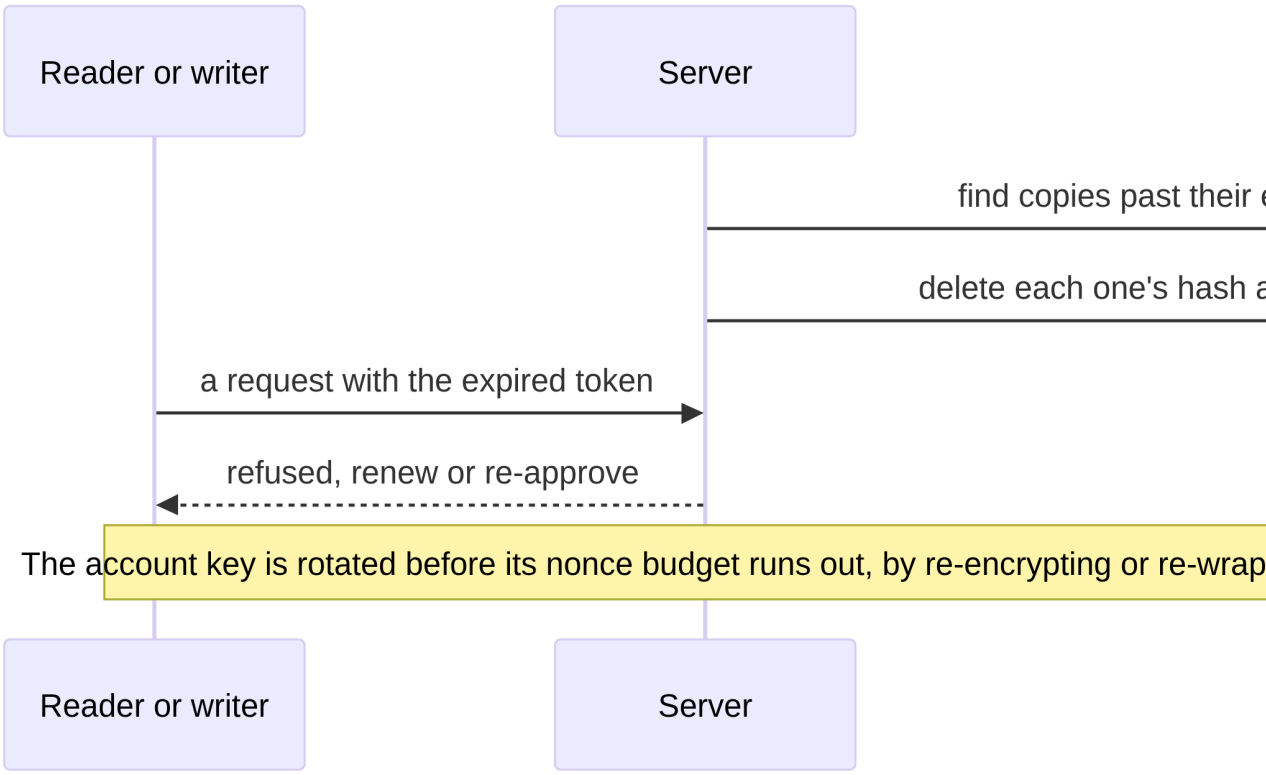
When the user approves a new reader or writer, the approval happens inside a session that already holds the account key. The server makes a new token, wraps the account key under a key derived from it, stores that copy beside the new token's hash, and hands the new token to the new holder once. Renewing a token follows the same steps, then deletes the old token's hash and copy. The account key itself does not change.



Why this is best practice. Granting access means wrapping the same key once more, not re-encrypting data or sharing one credential among holders. This is envelope encryption, the pattern cloud key-management services use, and it means every holder can be revoked on its own. Handing a new token over once and rotating it on renewal follows the OAuth security best current practice (RFC 9700).

9.1.4 Expire

Every token has an end time, stored beside its hash. The server refuses an expired token on every request, and a periodic sweep then deletes the expired token's hash and its copy of the account key. A holder renews before its end time by presenting its current token, which follows Update; a refresh token presented twice revokes its whole chain. The lifetime for each kind of holder is question 4. Backups can still hold an expired copy for up to 30 days (see What it does not protect).



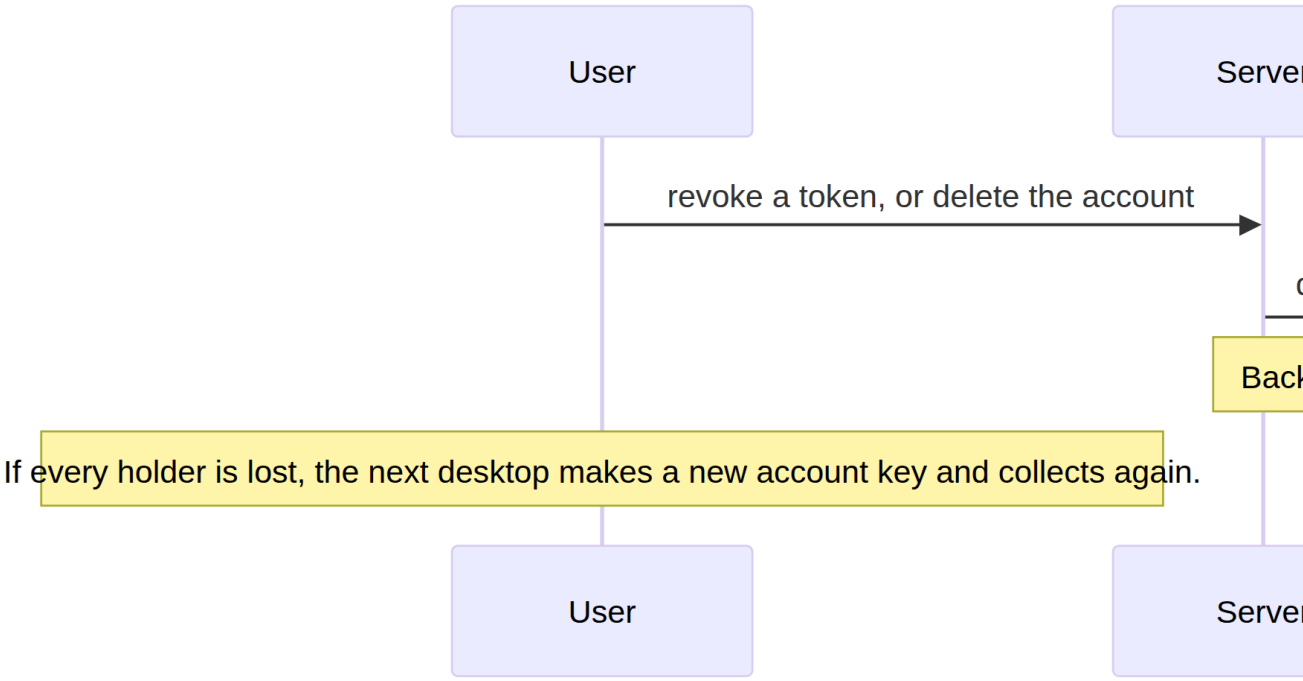
Why this is best practice. Every token has a fixed lifetime, and its copy of the key is deleted when that lifetime ends, so a leaked token stops working without anyone acting. Bounded lifetimes are what RFC 9700 recommends for tokens, and NIST calls them cryptoperiods (SP 800-57 Part 1).

9.1.5 Delete

The user ends access in one of three ways, each from an approved device:

- **Revoke one reader or writer** (an AI, a browser session, a script). The server deletes that token's hash and its copy of the account key. That holder is cut off at once, and every other holder keeps working.
- **Remove a lost device.** The same deletion, for the device's token. Recovery then runs as an Update from a device the user still has.
- **Delete the account.** The server deletes every copy and every row. With no copy of the account key left, nothing live can open the mirror.

Backups still hold old copies until they expire, up to 30 days. If every holder is lost, the next desktop makes a new account key and collects again from the sources.



Why this is best practice. Deleting every copy of the key, as account deletion does, is cryptographic erase, a sanitization method NIST recognizes (SP 800-88 Rev. 1). It works at once, even when ciphertext remains in storage the user cannot reach. Revoking a single holder is weaker: it ends that holder's access through the service, but other copies of the same key remain. Revocation takes effect server-side on the token's next use, as OAuth token revocation specifies (RFC 7009). The known exception is backups, which keep old copies for up to 30 days.

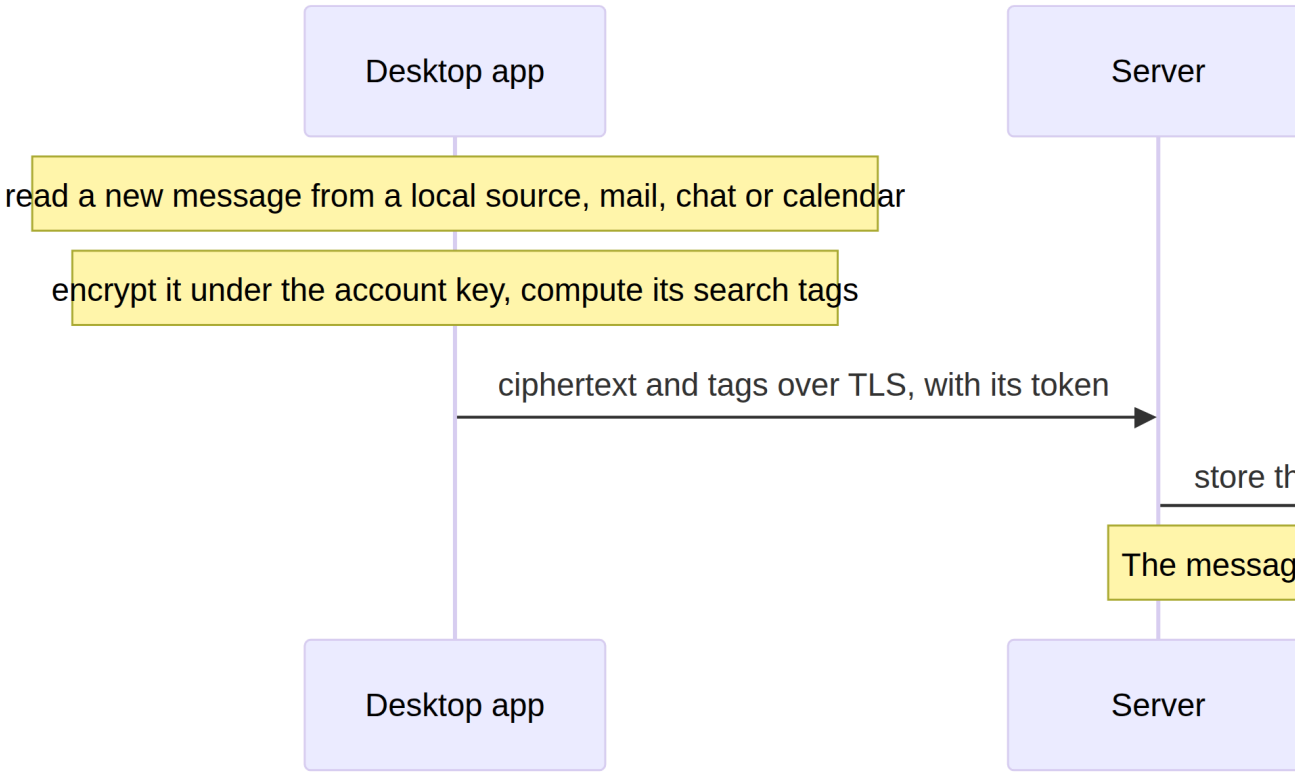
9.2 How data moves

Five flows cover everything the system does. In each one, the account key is opened only inside a request that presents an approved token.

9.2.1 Collect and push

New messages are locked on the user's own computer before they go anywhere, so the service stores only ciphertext, which opens only inside an approved request.

1. The desktop app reads a new message from a source on the desktop: mail, messaging apps, team chat, a calendar.
2. It encrypts the message under the content key and computes its search tags under the tag key, both derived from the account key.
3. It uploads the ciphertext and tags over TLS, presenting its own token. The server stores them and cannot open them without an approved token.
4. The message is searchable as soon as it is stored.



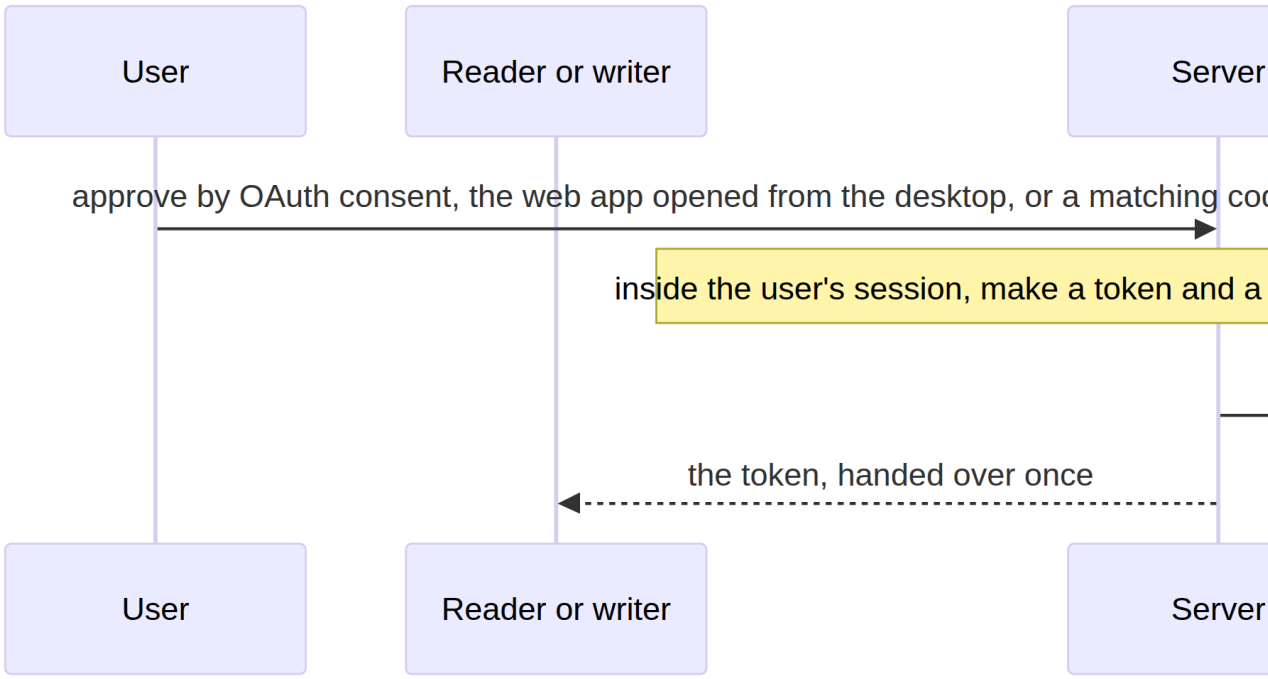
Only ciphertext and tags leave the desktop; the server cannot open them without an approved token.

Why this is best practice. Encrypting on the client before upload, so the storage provider holds only ciphertext, is the model behind end-to-end encrypted storage and client-side encryption. Each message is sealed with AES-256-GCM (NIST SP 800-38D), an authenticated cipher that detects any change to the ciphertext, and it is bound to its account and message ids as associated data (RFC 5116), so a row cannot be moved to another account or message. TLS protects the upload in transit (RFC 8446).

9.2.2 Approve a reader or writer

Nothing gets access unless the user says yes, and every approved AI, browser or device gets its own key that can be cancelled on its own.

1. The user approves it: OAuth consent for an AI, the web app opened from the desktop, or a matching code for a second device.
2. Inside the user's own approved session, the server makes a new token and a copy of the account key locked under it.
3. The token is handed to the new reader or writer once. The server keeps only its hash.



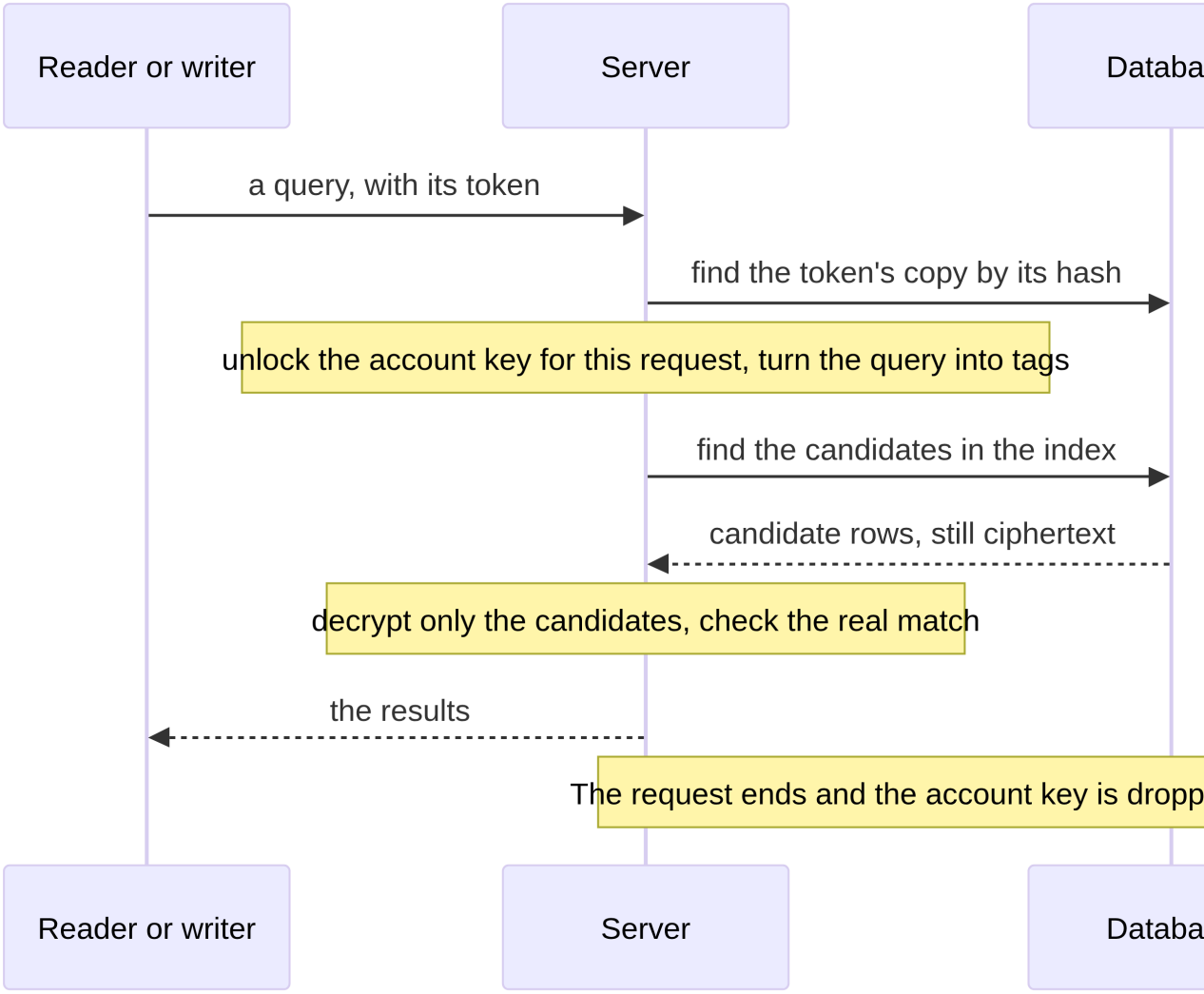
The server keeps only the token's hash; the token itself is handed over once.

Why this is best practice. AI clients are approved through OAuth consent with PKCE, as the OAuth security best current practice requires (RFC 9700), and a second device is approved by comparing a short code on both screens, a relative of the device flow in RFC 8628. Giving each holder its own token and its own wrapped copy of the key is least privilege: no credential is shared, so each one can be revoked alone. Storing only a hash of each token means a database copy yields no usable credential, as NIST SP 800-63B requires for stored passwords.

9.2.3 Search and retrieve

An approved AI gets answers in well under a second, and the data is unlocked only for that one question, then locked again.

1. A reader sends a query with its token.
2. The server finds the token's copy by its hash and unlocks the account key for this request.
3. It turns the query into tags, finds the candidates in the index, decrypts only those, and checks the real match.
4. It returns the results and drops the account key.



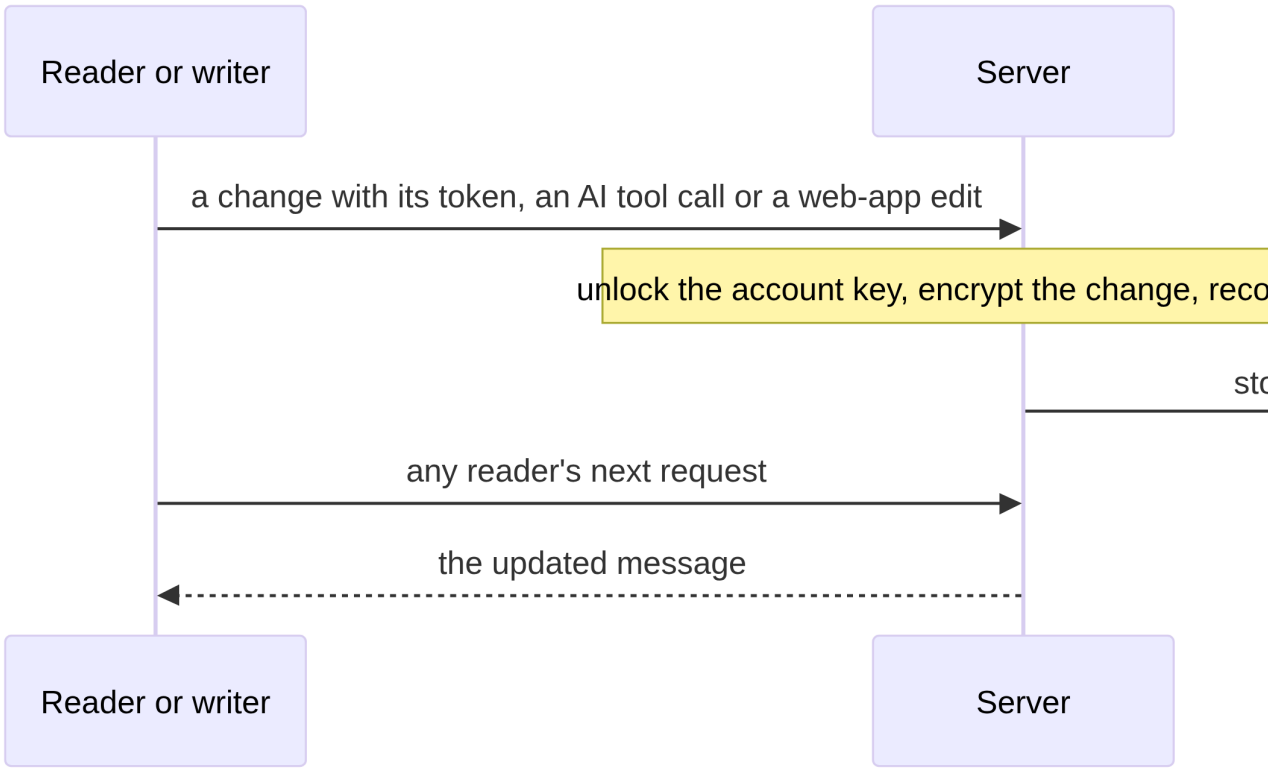
Only the candidate rows are decrypted, and the account key lives only for the request.

Why this is best practice. Search runs on keyed tags, the standard construction for searching encrypted data (Song, Wagner and Perrig, IEEE S&P 2000). The query is reduced to trigrams and the real match is checked afterwards, as in production regex search (Russ Cox, trigram index). Holding the key in memory for one request only follows NIST SP 800-57 Part 1 on limiting how long plaintext keys exist. The known cost is access-pattern leakage (Cash et al., CCS 2015), which question 1 puts to reviewers.

9.2.4 Update

Edits by an approved AI or by the user are locked the moment they are saved, and every other reader sees them on its next request.

1. A writer sends a change with its token: an AI's tool call, or an edit on the web app.
2. The server unlocks the account key, encrypts the change, recomputes its tags and stores it.
3. Every reader sees it on its next request.



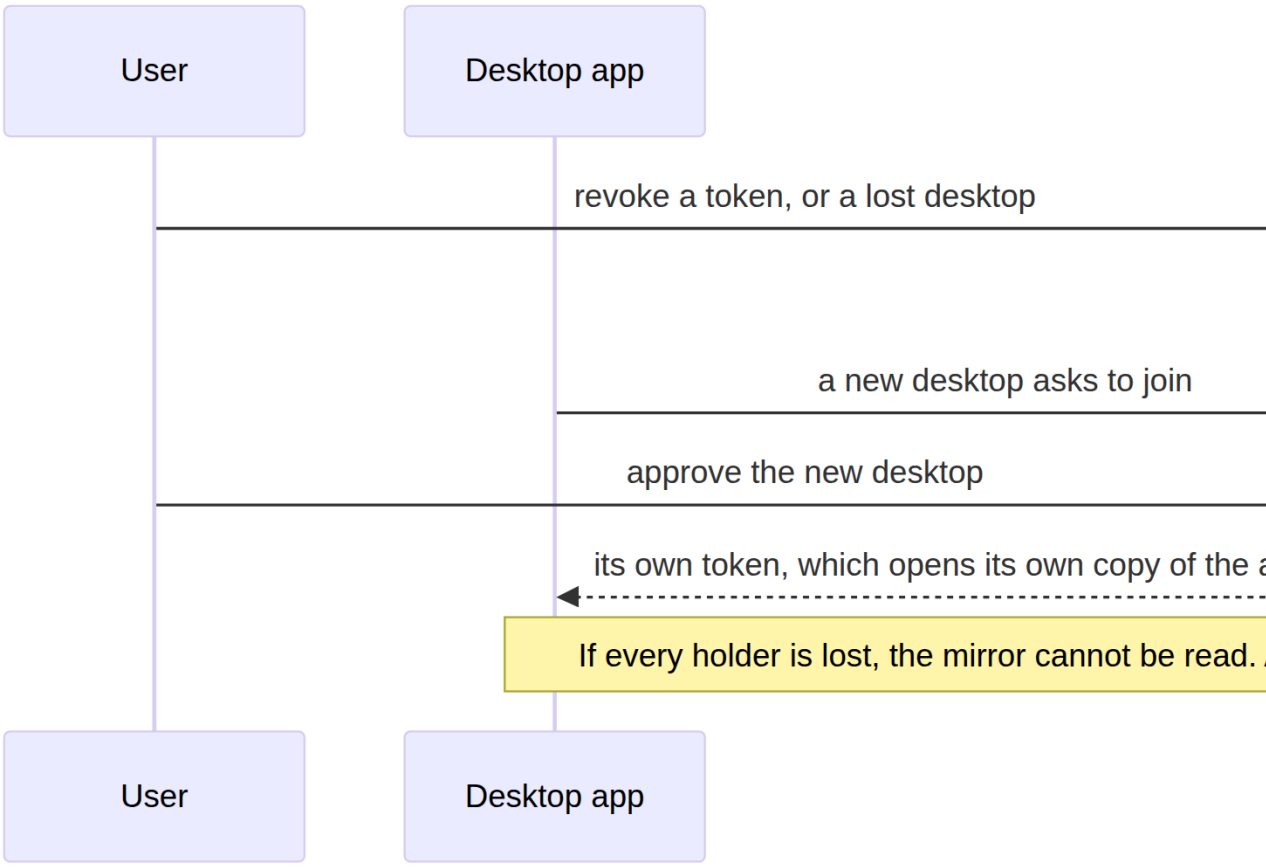
Writes follow the same rule as reads: the account key is opened only inside the writer's request.

Why this is best practice. A write uses the same authenticated encryption as a read, so the server never stores an edited row in the clear. Every write gets a fresh random nonce, which is what keeps AES-GCM secure when the same key encrypts many messages (NIST SP 800-38D, section 8). The tags are recomputed with HMAC-SHA256 (FIPS 198-1), so the search index never falls out of step with the content.

9.2.5 Revoke and recover

Cutting off an AI or a lost device takes effect at once and touches no one else; a replacement device is approved like any other.

- **Revoke.** Deleting a token's hash and copy cuts that reader or writer off at once.
- **A lost desktop.** From another approved device the user revokes it, then approves a new desktop, which receives the account key through its own copy.
- **The only desktop lost.** A recovery code made at setup is one more token, printed and kept offline, with its own locked copy. Presented from a new desktop, it approves that desktop. Without one, the mirror starts again with a new account key even if AI readers still hold tokens, because a reader's token cannot approve a device.
- **Every holder lost.** The mirror cannot be read. A new desktop makes a new account key and collects again from the sources.

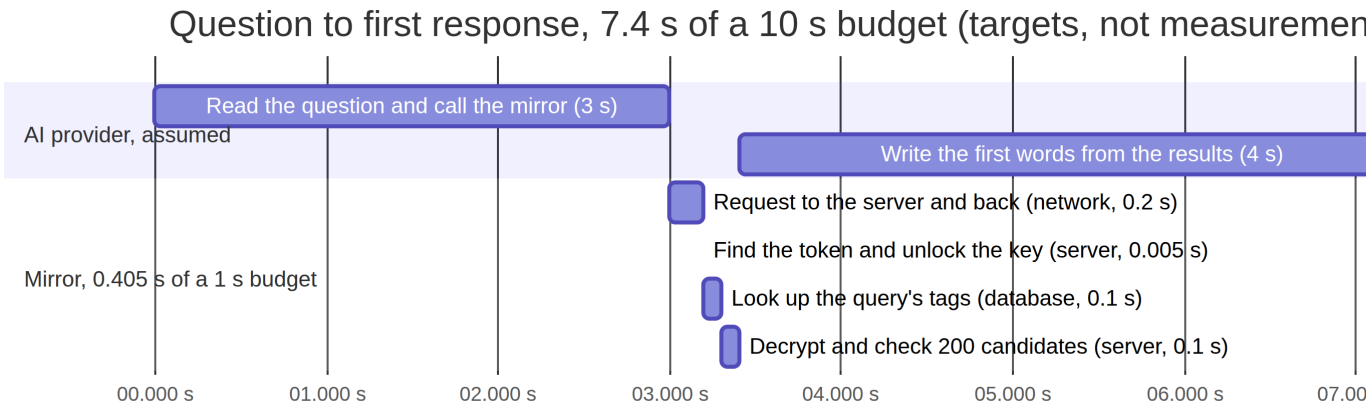


Cutting off a reader is one deletion; a replacement desktop is approved like any other holder.

Why this is best practice. Revoking a token server-side, so that it fails on its next use, is what OAuth token revocation specifies (RFC 7009). Deleting that token's copy ends its access through the service. Only deleting every copy is cryptographic erase, the sanitization method NIST recognizes (SP 800-88 Rev. 1). Recovering through another approved device, rather than through a reset the operator controls, keeps the operator unable to open the data. End-to-end encrypted services use the same approach, and when every holder is lost the honest result is that the data cannot be read.

10. Latency budget

Encryption costs the answer well under a second: the mirror's whole share is targeted at 0.4 seconds, inside a 1-second budget, so most of the 10 seconds belongs to the AI.



These are targets, not measurements. Before launch they are measured on a synthetic account the size of the largest real one, about 1.6 million messages. A new message is searchable once its upload completes; the target is under 2 seconds from arriving on the desktop.

11. What it does not protect

This is encryption controlled by the user's keys, not end-to-end encryption: the server decrypts inside each approved request. The limits follow.

Limit	Consequence
The server holds the account key and the results while it answers	A compromised server can capture the account key from any approved request. That one key opens the whole mirror, past and future, until the key is rotated. Proving which code the server runs (attestation) would close this; it is not in this design.
AI providers see what they retrieve	Their handling of results is outside this design
Search tags reveal which messages share a fragment	The tags are deterministic and overlap, so a thief of the database may recover text from fragment frequencies alone, and more with some known plaintext (question 1). Sizes and times are visible too.
A stolen token works until it is revoked	Expiry limits the damage (question 4)
Malware on the desktop sees everything	The desktop holds the account key
Backups keep deleted copies until they expire	A revoked token can still open its copy in a backup taken earlier, for up to 30 days
Revoking one holder is access control, not erasure	Other copies of the same key remain, and a holder that paired its token with a copy of the database keeps the key. Only deleting every copy, or rotating the key, erases.
Every token opens the whole account key	What a holder may do (read only, one source, approve others) is enforced by the server's rules, not by cryptography
Tokens, queries and results are plaintext in the server's memory	They must be kept out of logs, traces and error reports. Whatever ends TLS in front of the server sees tokens too. Mitigation: the implementation's source code is published, and automated tests plant marker values in tokens, queries and results and fail if a marker reaches a log, a trace, an error report or stored data. Publishing the source shows what the code does; proving that the running server is that code is what attestation would add (question 7).
The mirror is only as fresh as the desktop	While the desktop is off or asleep, nothing new is collected
Search needs a fixed fragment of three or more characters	A query without one cannot use the index. It must be refused, or answered by decrypting every row.
Attachments are encrypted by the storage service, not on the desktop	The storage service sees each file in the clear while it handles an upload or a read. At rest, a file opens only with the storage key, which is derived

Limit	Consequence
	from the account key inside an approved request. The bucket's access policy stays as a second line.

12. Decisions made to simplify

These choices keep the design small. Each has a cost, stated here so reviewers can challenge it.

Decision	What it removes	What it costs
Every source is collected on the desktop	Server-side collectors, and the public-key sealing they needed for mail that arrives while nobody is calling	A source that can only be read server-side, such as a hosted inbox or a chat bot, is out of scope until it can run on the desktop
The token is its own secret	Separate unlock secrets. SHA-256 of the token finds its copy; a key derived from the token with HKDF opens it, and knowing one gives nothing about the other.	A leaked token both identifies and unlocks, exactly as a leaked token-and-secret pair did before
Support is just another reader	The operator's hardware-held support key. To get help, the user approves a support reader with an end time, like any other.	Its use is recorded in the service's own log rather than by the cloud provider's audit log
Background AI work needs a reader	Server-side processing with no request open. Triage or filing runs on the desktop, or inside a reader's request.	Nothing is processed while every reader is idle
Attachments are encrypted by the storage service, under a key derived from the account key	Encrypting and chunking every file on the desktop	The storage service sees each file in the clear while it handles a request, and it must accept a key supplied per request. The bucket cannot hand out direct links, because every read needs the key.

13. Questions for reviewers

These seven are where review is wanted most.

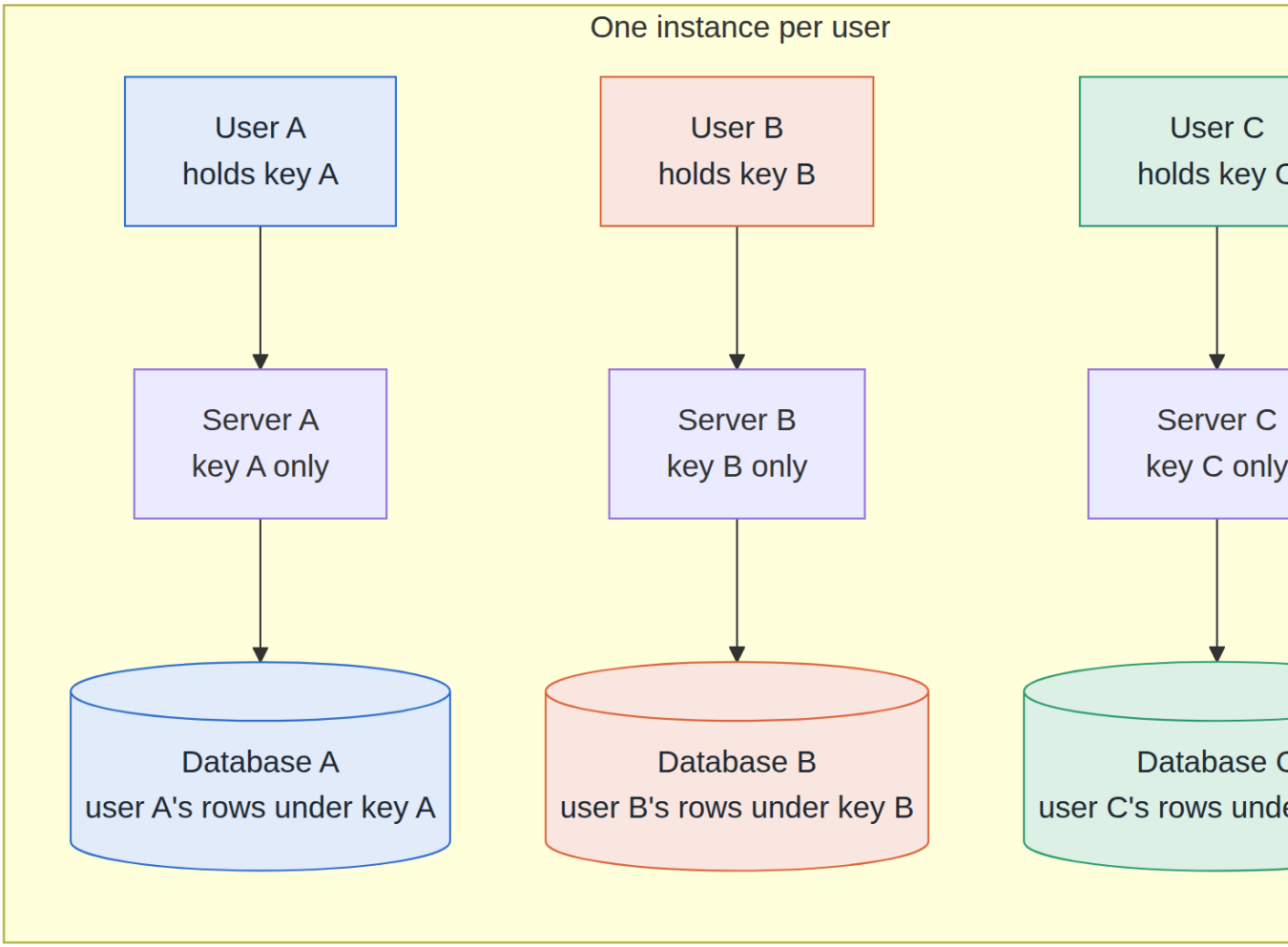
1. **Search leakage.** Is a keyed trigram index acceptable for this threat model, or should tag sets be padded?
2. **Nonce budget.** One content key encrypts every message for the account's life with random 96-bit nonces, and NIST SP 800-38D limits that to 2^{32} uses per key. The largest account today holds about 1.6 million messages, far below that. Proposed answer: derive a new content key each year with HKDF from the account key and the year, so the limit applies per year and nothing extra is stored. Is that enough, or is a cipher that tolerates nonce reuse (AES-GCM-SIV, XAES-256-GCM) still worth adding?
3. **Rotating the account key.** Proposed answer: rotation is a re-push. The desktop makes a new account key, re-encrypts the mirror under it, including what writers added, uploads it again and deletes the old copies. That is the same path as when every holder is lost. It costs a full upload, so it is for a suspected compromise, not a schedule. Is that acceptable, or are data keys wrapped by the account key worth their extra machinery?
4. **Token lifetimes.** What expiry and renewal should each kind have: desktop, AI access and refresh, web app session, script?
5. **A second desktop.** Is a matching code shown on both screens enough to approve one? And how should the new desktop's token reach it without the server storing it: handed to a waiting request in memory, or sealed to a key the new desktop sends?
6. **Support audit.** Is the service's own log enough for support access, or is an independent audit worth a second key?
7. **Attestation.** Is proving which code the server runs worth adding later, to close the first limit?

Appendices

A. Appendix A. Single or multi-tenant

For stored data the choice barely matters, because each user's rows are locked under that user's own key. It matters for the server's memory, for cost, and for who can see metadata.

	One shared database	One database per user
Stored data	Isolated by each user's key: another user's rows, served by mistake, do not decrypt	The same, plus physical separation
A compromised server	Sees what every active user's requests decrypt while it is compromised	Sees one user's
Metadata: counts, sizes, times	Visible to the operator. Per-user tag keys stop matching across users.	Visible to whoever runs it
Deleting a user	Delete their copies of the account key and their rows	Delete the database
Cost and operations	One database to run	One per user



The recommendation is one codebase that runs either way. Shared by default, with row-level security in the database as a second line behind the keys. A dedicated instance, run by the operator or by the user, for anyone who wants the smaller blast radius. The keys stay with the user in both.